Agent Behavior Evaluation

GitHub repository link : https://github.com/nuzaeromar/AI-Augmented-SE.git

Across the three tasks, there was a clear mismatch between what the prompts were asking the agent to do and what the system can actually save to disk. When the prompt pushed it to create a multi-file project, the model often responded by describing a folder structure, like src/app.py, /catalog.py, either as a dictionary or as separate file blocks. But the create_program() pipeline doesn't know how to turn that kind of response into real files. It only writes one file to module_path, and it doesn't parse JSON scaffolds or split text into multiple files. In other words, the agent hallucinates files not because it's making up random filenames, but because the current workflow has no mechanism to convert a multi-file description into actual separate ones.

The agent is consistent about creating a timestamped repo under output/<name>_<timestamp>/, writing to src/main.py by default, and blocking unsafe paths via Tools._safe(), but the generated code often doesn't match how Python modules work in a src/layout (e.g., using from catalog import ... without proper package context). More importantly, even when the model describes a multi-file src/project, the single-file write behavior means that structure never actually appears on disk. Constraint-following is also uneven. The code added reflection text like === SELF-REVIEW === which turns into a syntax error because the agent writes the entire response as Python source. The agent also tended to generate unnecessary code relative to the task requirements. The addition of a self-review block is another example of unnecessary content for a runnable artifact. In general, the system produced more surface area than required, and the extra output increased the probability of violations (syntax errors, import problems, and environment-dependent failures).

Prompt engineering attempts improved clarity but also surfaced the limits of prompt-only solutions. The persona pattern ("senior Python engineer") helped establish expectations about correctness and runnability, and the decomposition pattern made the intended workflow explicit. The reflection pattern was introduced to encourage self-checking, but because the agent writes model output directly into a Python file, reflection text must be either removed before writing or emitted in a different channel or format that tooling can strip. In practice, reflection improved thinking, but it harmed executability unless the agent's logic was updated to separate reflection from code. The biggest lesson from the before/after comparison is that prompt improvements cannot compensate for a missing capability in the agent.

The most impactful improvement going forward is to align tooling with the multi-file prompting strategy. The agent needs a scaffold mode that explicitly requests a JSON scaffold (e.g., {"files":[{"path":...,"content":...}]}), parses it safely. The CLI should expose this as a separate command (e.g., cca scaffold) so multi-file generation is opt-in and does not break the single-file create path. Finally, as more agents are built, it would be valuable to introduce lightweight semantic checks (forbidden imports per task, consistent ID types across modules, import resolution under src/ layout) and to keep reflection out-of-band so it improves quality without breaking executability.